# C언어 자료구조와 알고리즘

**- 확인 및 시험**

# 목차

01 자료구조·알고리즘·추상 자료형 3
02 C언어 메모리·배열·구조체·포인터 4
03 알고리즘 성능 분석 5
04 순환과 재귀 6
05 연결 리스트 7
06 스택·수식·미로 8
07 큐·덱·시뮬레이션 9
08 트리·순회·트리 응용 10
09 이진 탐색 트리·균형 트리 11
10 우선순위 큐·힙·허프만 12
11 그래프·순회·위상 정렬·분리 집합 13
12 최소 비용 신장 트리·최단 경로 14
13 정렬 15
14 탐색 16
15 해싱·해시 테이블 17
16 문자열 탐색 18
17 분할 정복 19
18 동적 계획법 20
19 탐욕 알고리즘 21
20 백트래킹 22

# 01 자료구조·알고리즘·추상 자료형

## 핵심 정의

**자료구조**: 자료들을 정리하여 보관하는 여러 가지 구조를 **자료구조**(data structure)라고 부른다. [U01-001] (A 1.1, p.12)

**알고리즘**: **알고리즘**은 컴퓨터로 문제를 풀기 위한 단계적인 절차이다. [U01-003] (A 1.1, p.15)

**추상 자료형**: **추상 자료형**(ADT: abstract data type)은 추상적·수학적으로 자료형을 정의한 것이다. [U01-002] (A 1.2, p.18)

## ADT와 구현의 구분

| 구분 | 책의 설명 |
| --- | --- |
| **추상 자료형** [U01-032] (A 1.2, p.19) | ADT는 데이터나 연산이 무엇(what)인지 정의하며, 어떻게(how) 구현할 것인지는 정의하지 않는다. [U01-019] (A 1.2, p.19) |
| **자료구조** [U01-033] (A 1.2, p.18) | **자료구조**는 **추상 자료형**을 프로그래밍 언어로 구현한 것이라 할 수 있다. [U01-017] (A 1.2, p.18) |

# 02 C언어 메모리·배열·구조체·포인터

### 핵심 정의

**배열**: **배열**은 동일한 타입의 데이터를 여러 개 만들 때 사용하며, 연속적인 메모리 공간에 인덱스로 접근한다. [U02-001] (A 3.1, p.70)

**포인터**: **포인터**(pointer)는 다른 변수의 주소를 가지고 있는 변수이다. [U02-017] (A 3.5, p.89)

**동적 메모리**: **동적 메모리 할당**은 필요한 메모리를 운영체제로부터 할당받아 사용하고, 사용이 끝나면 시스템에 반납하는 기능이다. [U02-002] (A 3.6, p.95)

### A의 배열·구조체 구분

| 구분 | 원문 정의 |
| --- | --- |
| **배열** [U02-038] (A 3.2, p.73) | **배열**은 동일한 타입의 데이터를 여러 개 만들 때 사용하며, 연속적인 메모리 공간에 인덱스로 접근한다. [U02-001] (A 3.1, p.70) |
| **구조체** [U02-039] (A 3.2, p.73) | **배열**이 같은 타입의 데이터 모임이라면 **구조체**(structure)는 타입이 다른 데이터를 묶는 방법이며 struct로 표기한다. [U02-007] (A 3.2, p.73) |

### 주요 수식·복잡도

**list[i] 요소의 주소**: base+i*sizeof(int); base는 list[0]의 주소이다. [U02-037] (A 3.1, p.72)

# 03 알고리즘 성능 분석

## 핵심 정의

**T(n)**: 연산의 수행 횟수를 입력 개수 n의 함수로 나타낸 것을 **시간 복잡도 함수** T(n)이라고 한다. [U03-009] (A 1.3, p.25)

**재귀 방정식**: **재귀 방정식**은 자기 자신을 항으로 갖는 방정식이며, 재귀 **알고리즘**의 수행 시간을 나타내는 데 사용한다. [U03-003] (B 11.3.1, p.508-509)

**마스터 정리**: **마스터 정리**는 $T(n)=aT(n/b)+f(n)$ 형태의 **재귀 방정식**에 대한 점근적 한계를 구하는 방법이며, $a \geq 1$, $b>1$은 상수이다. [U03-032] (B 11.3.3, p.517)

## A의 점근 표기법

| 표기법 | 원문 설명 |
| --- | --- |
| $O(g(n))$ [U03-039] (A 1.3, p.28) | 모든 $n>n_0$에 대해 $\|f(n)\| \leq c\|g(n)\|$을 만족하는 상수 $c$와 $n_0$가 존재하면 $f(n)=O(g(n))$이다. [U03-013] (A 1.3, p.28) |
| $\Omega(g(n))$ [U03-040] (A 1.3, p.32) | **빅오메가 표기법**은 함수의 하한을 나타내며, 모든 $n>n_0$에 대해 $\|f(n)\| \geq c\|g(n)\|$을 만족하는 상수 $c$와 $n_0$가 존재하면 $f(n)=\Omega(g(n))$이다. [U03-017] (A 1.3, p.32) |
| $\Theta(g(n))$ [U03-041] (A 1.3, p.32) | **빅세타 표기법**은 함수의 상한과 하한을 동시에 나타내며, 모든 $n>n_0$에 대해 $c_1\|g(n)\| \leq \|f(n)\| \leq c_2\|g(n)\|$을 만족하는 상수 $c_1$, $c_2$, $n_0$가 존재하면 $f(n)=\Theta(g(n))$이다. [U03-018] (A 1.3, p.32) |

## 주요 수식·복잡도

**순차 탐색 평균 비교 횟수**: $(1+2+\cdots+n)/n=(n+1)/2$; 각 항목이 **탐색**될 확률이 $1/n$으로 동일하다는 가정이다. [U03-037] (A 1.3, p.35)

# 04 순환과 재귀

## 핵심 정의

**순환**: **순환**(recursion)은 어떤 **알고리즘**이나 함수가 자기 자신을 호출하여 문제를 해결하는 프로그래밍 기법이다. [U04-001] (A 2.1, p.40)

**활성 레코드**: **활성 레코드**(activation record)는 함수를 위해 시스템 스택에서 사용하는 공간이다. [U04-012] (A 2.1, p.44)

**꼬리 순환**: **꼬리 순환**(tail recursion)은 **순환** 호출이 **순환** 함수의 맨 끝에서 이루어지는 형태이며, 쉽게 **반복** 형태로 변환할 수 있다. [U04-028] (A 2.1·2.4, p.47,62)

## 원문의 계산 방법별 시간 복잡도

| 계산 방법 | 원문 결과 |
|---|---|
| 팩토리얼: **반복·순환** [U04-041] (A 2.1, p.50) | 팩토리얼의 **반복 알고리즘**과 **순환 알고리즘**은 모두 **시간 복잡도**가 O(n)이다. [U04-016] (A 2.1, p.50) |
| 거듭제곱: **반복** [U04-042] (A 2.2, p.53) | **반복**적인 slow_power()는 루프마다 한 번의 곱셈이 필요하고 루프 횟수가 n이므로 **시간 복잡도**는 O(n)이다. [U04-019] (A 2.2, p.53) |
| 거듭제곱: **순환** [U04-043] (A 2.2, p.53) | **순환**적인 거듭제곱 계산은 호출할 때마다 문제 크기가 약 절반으로 줄어 **시간 복잡도**가 $O(\log_2 n)$이다. [U04-003] (A 2.2, p.52-53) |

## 주요 수식·복잡도

**순환적인 피보나치 계산의 상한**: $O(2^n)$ [U04-040] (A 2.3, p.55)

# 05 연결 리스트

## 핵심 정의

**리스트**: **리스트**의 객체는 n개의 element형으로 구성된 순서 있는 모임이다. [U05-003] (A 6.1, p.177)

**헤드 포인터**: **헤드 포인터**(head pointer)는 연결 **리스트**의 첫 번째 **노드**를 가리키는 변수이다. [U05-013] (A 6.3, p.185)

**헤드 노드**: **헤드 노드**(head node)는 데이터를 가지고 있지 않은 특별한 **노드**로 삽입·삭제 **알고리즘**을 간편하게 한다. [U05-027] (A 7.3, p.230)

## 연결 리스트 종류

| 항목 | 원문 설명 |
| --- | --- |
| 단순 연결 **리스트** [U05-040] (A 6.3, p.186) | 단순 연결 **리스트**(singly linked list)는 하나의 방향으로만 연결되며 체인(chain)이라고도 한다. [U05-023] (A 6.3, p.186) |
| **원형 연결 리스트** [U05-041] (A 6.3, p.186) | **원형 연결 리스트**(circular linked list)는 마지막 **노드**의 링크가 첫 번째 **노드**를 가리킨다. [U05-025] (A 6.3, p.186) |
| 이중 연결 **리스트** [U05-042] (A 6.3, p.186) | 이중 연결 **리스트**(doubly linked list)는 각 **노드**에 앞 **노드**와 뒤 **노드**를 가리키는 2개의 링크가 존재한다. [U05-002] (A 6.3, p.186) |

## 주요 수식·복잡도

**배열 리스트의 접근**: get_entry: O(1); **배열**의 인덱스로 항목에 바로 접근한다. [U05-038] (A 6.2, p.183)

**배열 리스트의 삽입·삭제**: 최악의 경우 O(n); 맨 끝 삽입 O(1); 다른 항목을 **이동**하는 삽입·삭제의 최악 경우와 맨 끝 삽입을 구분한다. [U05-039] (A 6.2, p.183)

# 06 스택·수식·미로

## 핵심 정의

**스택**: 스택의 입출력 형태는 **후입선출**(LIFO: Last-In First-Out)이다. [U06-001] (A 4.1, p.102)

**후위 표기법**: 후위(postfix)는 연산자가 피연산자 뒤에 있는 표기법이다. [U06-002] (A 4.5, p.124)

**토큰**: 텍스트 분석에서 **토큰**은 최소 단위의 기호 또는 단어를 말한다. [U06-026] (B 2.4.2, p.96)

## 기본 연산

| 항목 | 원문 설명 |
| --- | --- |
| push [U06-038] (A 4.1, p.105) | push는 스택의 맨 위에 item을 추가하는 연산이다. [U06-007] (A 4.1, p.105) |
| pop [U06-039] (A 4.1, p.105) | pop은 스택의 맨 위의 원소를 제거해서 반환하는 연산이다. [U06-008] (A 4.1, p.105) |
| peek [U06-040] (A 4.1, p.105) | peek는 스택의 맨 위의 원소를 제거하지 않고 반환하는 연산이다. [U06-009] (A 4.1, p.105) |

## 주요 수식·복잡도

**A의 배열 스택 상태 조건**: 공백: top == -1; 포화: top >= (MAX_STACK_SIZE-1); **배열** 인덱스는 0부터 시작한다. [U06-037] (A 4.2, p.107)

# 07 큐·덱·시뮬레이션

### 핵심 정의

**큐**: 큐(queue)는 먼저 들어온 데이터가 먼저 나가는 선입선출(FIFO: First-In First-Out) 구조이다. [U07-001] (A 5.1, p.146)

**덱**: 덱(deque)은 double-ended queue의 줄임말이며 전단과 후단에서 모두 삽입·삭제가 가능한 큐이다. [U07-002] (A 5.5, p.162)

**연결된 큐**: **연결된 큐**에 요소가 없으면 front와 rear는 NULL이다. [U07-024] (A 7.5, p.243)

### A·B 원형 구현의 차이

| 항목 | 원문 설명 |
|---|---|
| A **원형큐** [U07-037] (A 5.3, p.154) | A의 **원형큐**에서 front는 첫 번째 요소의 하나 앞을, rear는 마지막 요소를 가리킨다. [U07-011] (A 5.3, p.154) |
| B **순환** 큐 [U07-038] (B 3.2.2, p.124) | B의 **순환** 큐에서 Front는 제거할 전단의 위치를, Rear는 실제 후단 위치에 1을 더한 값을 나타낸다. [U07-017] (B 3.2.2, p.124-125) |

### 주요 수식·복잡도

**A의 공백·포화 조건**: 공백: front == rear; 포화: (rear+1) % MAX_QUEUE_SIZE == front; 한 자리를 비워 두는 원문 구현이다. [U07-036] (A 5.3, p.158)

# 08 트리·순회·트리 응용

## 핵심 정의

**이진트리**: **이진트리**는 공집합이거나 **루트**와 왼쪽·오른쪽 서브 트리로 구성된 유한 집합이며 서브 트리들도 **이진트리**여야 한다. [U08-001] (A 8.2, p.258)

**포화 이진 트리**: **포화 이진 트리**(full binary tree)는 각 레벨에 **노드**가 꽉 차 있는 **이진트리**이다. [U08-015] (A 8.2, p.261)

**완전 이진 트리**: **완전 이진 트리**(complete binary tree)는 마지막 전 레벨까지 모두 채워지고 마지막 레벨은 왼쪽부터 순서대로 채워진다. [U08-016] (A 8.2, p.262)

## V=루트, L=왼쪽, R=오른쪽

| 항목 | 원문 설명 |
|---|---|
| 전위**순회** [U08-049] (A 8.4, p.268) | 전위**순회**(preorder traversal)의 방문 순서는 VLR이다. [U08-022] (A 8.4, p.268) |
| 중위**순회** [U08-050] (A 8.4, p.268) | 중위**순회**(inorder traversal)의 방문 순서는 LVR이다. [U08-023] (A 8.4, p.268) |
| 후위**순회** [U08-051] (A 8.4, p.268) | 후위**순회**(postorder traversal)의 방문 순서는 LRV이다. [U08-024] (A 8.4, p.268) |

## 주요 수식·복잡도

**A의 배열 표현 인덱스**: 부모: i/2; 왼쪽 자식: 2i; 오른쪽 자식: 2i+1; **루트** 인덱스 1인 원문 **배열** 표현. [U08-048] (A 8.3, p.264)

# 09 이진 탐색 트리·균형 트리

## 핵심 정의

**이진 탐색 트리의 키**: 왼쪽 서브 트리의 키들은 **루트** 키보다 작고 오른쪽 서브 트리의 키들은 **루트** 키보다 크다. [U09-004] (A 8.11, p.295)

**AVL 트리**: **AVL 트리**는 각 **노드**에서 왼쪽과 오른쪽 서브 트리의 높이 차이가 1 이하인 **이진 탐색 트리**이다. [U09-002] (A 13.5, p.511)

**레드 블랙 트리**: **레드 블랙 트리**는 **노드**를 빨간색 또는 검은색으로 표시하여 트리 균형을 유지하는 **이진 탐색 트리**이다. [U09-003] (B 6.5, p.267)

## 원문의 노드·색 구분

| 구분 | 내용 |
| --- | --- |
| **2-노드** [U09-078] (A 13.6, p.523) | **2-노드**는 데이터 k1 하나와 자식 **노드** 둘을 가진다. [U09-030] (A 13.6, p.523) |
| **3-노드** [U09-079] (A 13.6, p.523) | **3-노드**는 데이터 k1, k2 둘과 자식 **노드** 셋을 가진다. [U09-031] (A 13.6, p.523-524) |
| **4-노드** [U09-080] (A 13.7, p.528) | **4-노드**는 데이터 셋과 자식 넷을 가질 수 있다. [U09-037] (A 13.7, p.528) |

## 주요 수식·복잡도

**이진 탐색 트리 연산**: O(h); h는 트리 높이이다. 경사 트리의 최악 경우는 h=n, O(n)이다. [U09-070] (A 8.11, p.309)

# 10 우선순위 큐·힙·허프만

## 핵심 정의

**우선순위 큐**: **우선순위 큐**는 우선순위 속성을 갖는 데이터의 삽입(Enqueue)과 제거(Dequeue) 연산을 지원하는 ADT이다. [U10-001] (B 7.1, p.305)

**최대 히프**: **최대 히프**(max heap)는 부모 **노드**의 키 값이 자식 **노드**의 키 값보다 크거나 같은 완전 **이진트리**이다. [U10-010] (A 9.3, p.328)

**최소 히프**: **최소 히프**(min heap)는 부모 **노드**의 키 값이 자식 **노드**의 키 값보다 작거나 같은 완전 **이진트리**이다. [U10-011] (A 9.3, p.328)

## 두 책의 배열 시작 위치

| 항목 | 원문 설명 |
|---|---|
| A [U10-054] (A 9.3, p.328) | A의 **배열** 표현은 인덱스 0을 사용하지 않고 **루트 노드**를 인덱스 1에 저장한다. [U10-014] (A 9.3, p.328) |
| B [U10-055] (B 7.2.3, p.312) | B의 **배열** 표현은 깊이 0의 뿌리 **노드**를 **배열**의 0번 요소에 저장한다. [U10-015] (B 7.2.3, p.312) |

## 주요 수식·복잡도

**A의 배열 인덱스 관계**: 왼쪽 자식 = 부모 인덱스*2; 오른쪽 자식 = 부모 인덱스*2+1; 부모 = 자식 인덱스/2; **루트** 인덱스 1을 사용하는 **배열** 표현. [U10-044] (A 9.3, p.329)

**B의 배열 인덱스 관계**: 왼쪽 자식: 2k+1; 오른쪽 자식: 2k+2; 부모: (k-1)÷2의 몫; k는 **노드**의 인덱스이며, **루트** 인덱스는 0. [U10-045] (B 7.2.3, p.313)

# 11 그래프·순회·위상 정렬·분리 집합

## 핵심 정의

**그래프**: **그래프**는 정점(vertex)과 간선(edge)들의 유한 집합이며 G = (V, E)로 표시한다. [U11-001] (A 10.2, p.363-364)

**네트워크**: 간선에 비용이나 가중치가 할당된 **그래프**를 가중치 **그래프**(weighted graph) 또는 **네트워크**(network)라 한다. [U11-006] (A 10.2, p.365)

**위상 정렬**: 방향 **그래프**의 각 정점들의 선행 순서를 위배하지 않으면서 모든 정점을 나열하는 것을 **위상 정렬**(topological sort)이라 한다. [U11-002] (A 11.7, p.431)

## 그래프 표현

| 항목 | 정의 |
|---|---|
| 인접 행렬 [U11-047] (A 10.3, p.369) | 인접 행렬(adjacency matrix)은 2차원 **배열**을 사용하여 **그래프**를 표현한다. [U11-015] (A 10.3, p.369) |
| 인접 **리스트** [U11-048] (A 10.3, p.373) | 인접 **리스트**(adjacency list)는 각 정점에 인접한 정점들을 연결 **리스트**로 표시한다. [U11-020] (A 10.3, p.373) |

## 주요 수식·복잡도

**무방향 완전 그래프의 간선 수**: n×(n-1)/2; n은 정점 수이다. [U11-045] (A 10.2, p.367)

**깊이 우선 탐색의 시간 복잡도**: 인접 리스트: O(n+e), 인접 행렬: O($n^2$); n은 정점 수, e는 간선 수이다. [U11-046] (A 10.5, p.385)

# 12 최소 비용 신장 트리·최단 경로

## 핵심 정의

**신장 트리**: **신장 트리**(spanning tree)는 **그래프** 내의 모든 정점을 포함하는 트리이다. [U12-003] (A 11.1, p.396)

**최소 비용 신장 트리**: **최소 비용 신장 트리**는 **신장 트리** 중 사용된 간선들의 가중치 합이 최소인 **신장 트리**이다. [U12-001] (A 11.1, p.397)

**최단 경로**: **최단 경로** 문제는 두 정점을 연결하는 경로 중 간선의 가중치 합이 최소가 되는 경로를 찾는 문제이다. [U12-002] (A 11.4, p.414)

## 최단 경로의 범위 — A

| 알고리즘 | 범위 |
|---|---|
| **Dijkstra** [U12-046] (A 11.5, p.415) | **Dijkstra 알고리즘**은 하나의 시작 정점으로부터 모든 다른 정점까지의 **최단 경로**를 찾는다. [U12-018] (A 11.5, p.415) |
| **Floyd** [U12-047] (A 11.6, p.425) | **Floyd 알고리즘**은 **그래프**의 모든 정점 사이의 **최단 경로**를 한 번에 모두 찾는다. [U12-027] (A 11.6, p.425) |

## 주요 수식·복잡도

**distance 갱신**: distance[w] = min(distance[w], distance[u]+weight[u][w]); u는 새로 추가된 정점이며 S에 속하지 않는 정점 w의 값을 갱신한다. [U12-041] (A 11.5, p.417)

**Floyd 갱신**: A[i][j] = min(A[i][j], A[i][k] + A[k][j]); k를 0부터 n-1까지 고려한다. [U12-042] (A 11.6, p.425)

# 13 정렬

## 핵심 정의

**정렬**: **정렬**(sorting)은 크기순으로 오름차순(ascending order)이나 내림차순(descending order)으로 나열하는 것이다. [U13-001] (A 12.1, p.442)

**안정성**: **안정성**(stability)은 동일 키값 레코드들의 상대적 위치가 **정렬** 후에도 바뀌지 않는 성질이다. [U13-002] (A 12.1, p.445)

**피벗 분할**: 퀵 **정렬**은 **피벗**(pivot)보다 작은 요소를 왼쪽, 큰 요소를 오른쪽으로 옮겨 비균등하게 **분할**한다. [U13-025] (A 12.7, p.470)

## A 표 12-1 발췌

| 알고리즘 | 최선 | 평균 | 최악 |
|---|---|---|---|
| 삽입 **정렬** [U13-052] (A 12.10, p.486) | O(n) [U13-053] (A 12.10, p.486) | O(n²) [U13-054] (A 12.10, p.486) | O(n²) [U13-054] (A 12.10, p.486) |
| 선택 **정렬** [U13-055] (A 12.10, p.486) | O(n²) [U13-054] (A 12.10, p.486) | O(n²) [U13-054] (A 12.10, p.486) | O(n²) [U13-054] (A 12.10, p.486) |
| 버블 **정렬** [U13-056] (A 12.10, p.486) | O(n²) [U13-054] (A 12.10, p.486) | O(n²) [U13-054] (A 12.10, p.486) | O(n²) [U13-054] (A 12.10, p.486) |
| 퀵 **정렬** [U13-057] (A 12.10, p.486) | O(nlog₂n) [U13-058] (A 12.10, p.486) | O(nlog₂n) [U13-058] (A 12.10, p.486) | O(n²) [U13-054] (A 12.10, p.486) |

## 주요 수식·복잡도

**기수 정렬의 시간 복잡도**: O(d·n); n은 정수 수, d는 각 정수의 자리수이다. [U13-051] (A 12.9, p.485)

# 14 탐색

## 핵심 정의

**탐색**: **탐색**(search)은 여러 자료 중에서 원하는 자료를 찾는 작업이다. [U14-001] (A 13.1, p.496)

**이진 탐색 전제**: **이진 탐색**을 적용하기 전에 **배열**이 반드시 **정렬**되어 있어야 한다. [U14-002] (A 13.3, p.501)

**보간 탐색**: **보간 탐색**(interpolation search)은 **탐색키**가 존재할 위치를 예측하여 **탐색**하는 방법이다. [U14-024] (A 13.3, p.507)

## B의 자기 구성법

| 방법 | 처리 |
|---|---|
| **전진 이동법** [U14-044] (B 6.2.1, p.237) | **전진 이동법**(Move To Front Method)은 **탐색**된 항목을 데이터의 가장 앞으로 옮긴다. [U14-012] (B 6.2.1, p.237) |
| **전위법** [U14-045] (B 6.2.2, p.239) | **전위법**(Transpose Method)은 **탐색**된 항목을 바로 이전 항목과 교환한다. [U14-013] (B 6.2.2, p.239) |
| **계수법** [U14-046] (B 6.2.3, p.241) | **계수법**(Frequency Count Method)은 각 요소의 **탐색** 횟수를 별도 공간에 저장하고 횟수가 높은 순으로 재구성한다. [U14-014] (B 6.2.3, p.241) |

## 주요 수식·복잡도

**성공 시 순차 탐색 평균 비교 횟수**: (1+2+3+...+n)/n = (n+1)/2; 모든 키가 **탐색**될 확률이 같다고 가정한다. [U14-037] (A 13.2, p.500)

**색인 순차 탐색의 복잡도**: O(m+n/m); m은 인덱스 테이블 크기, n은 주 자료 **리스트** 크기이다. [U14-038] (A 13.3, p.507)

# 15 해싱·해시 테이블

## 핵심 정의

**해싱**: **해싱**(hashing)은 키에 산술 연산을 적용하여 항목이 저장된 테이블 주소를 계산하고 접근하는 **탐색**이다. [U15-001] (A 14.1, p.534)

**충돌·동의어**: 서로 다른 키 k1, k2에 대해 h(k1)=h(k2)이면 **충돌**(collision)이며 두 키는 동의어(synonym)이다. [U15-011] (A 14.3, p.538)

**적재 밀도**: **적재 밀도**(loading density) 또는 적재 비율(loading factor)은 저장 항목 수 n과 테이블 크기 M의 비율이다. [U15-002] (A 14.7, p.564)

## 충돌 해결

| 방법 | 설명 |
| --- | --- |
| **개방 주소법** [U15-053] (A 14.5, p.547) | **개방 주소법**(open addressing)은 **충돌**이 일어난 항목을 테이블의 다른 위치에 저장한다. [U15-023] (A 14.5, p.547) |
| 이중 **해싱**법 [U15-056] (A 14.5, p.555) | A의 이중 **해싱**법(double hashing) 또는 재**해싱**(rehashing)은 다음 저장 위치를 별개의 **해시 함수**로 결정한다. [U15-029] (A 14.5, p.555) |
| **체이닝** [U15-057] (A 14.6, p.559) | **체이닝**(chaining)은 각 **버킷**에 연결 **리스트**를 할당하여 **오버플로우**를 해결한다. [U15-033] (A 14.6, p.559) |

## 주요 수식·복잡도

**제산 해시 함수**: h(k) = k mod M; M은 해시 테이블 크기이다. [U15-050] (A 14.4, p.543)

**적재 밀도**: α = n/M; n은 저장된 항목 수, M은 해시 테이블의 **버킷** 수이다. [U15-051] (A 14.7, p.564)

# 16 문자열 탐색

### 핵심 정의

**본문**: **본문**(Text)은 **탐색** 대상이 되는 문자열이다. [U16-001] (B 10.1, p.457)

**패턴**: **패턴**(Pattern)은 **탐색**어를 의미한다. [U16-003] (B 10.1, p.457)

**경계**: **경계**(Border)는 문자열에서 일치하는 접두부와 접미부 쌍이다. [U16-019] (B 10.4.1, p.472)

### 문자열 탐색의 원문 비교

| 항목 | 원문 설명 |
|---|---|
| 고지식한 **탐색** [U16-039] (B 10.2, p.458) | **본문** 문자열을 처음부터 끝까지 차례대로 **순회**하면서 **패턴** 내 문자들을 일일이 비교한다. [U16-006] (B 10.2.1, p.458) |
| 카프-라빈 **알고리즘** [U16-040] (B 10.3, p.463) | **패턴**의 해시값과 **본문** 내 하위 문자열의 해시값을 비교한다. [U16-010] (B 10.3.1, p.463) |
| KMP **알고리즘** [U16-041] (B 10.4, p.472) | KMP **알고리즘**은 **경계**를 활용해서 불필요한 문자 비교를 피한다. [U16-020] (B 10.4.1, p.472) |
| 보이어-무어 **알고리즘** [U16-042] (B 10.5, p.480) | 보이어-무어 **알고리즘**은 문자열을 오른쪽에서 왼쪽으로 비교하지만 **이동**은 왼쪽에서 오른쪽으로 이루어진다. [U16-023] (B 10.5, p.480) |

### 주요 수식·복잡도

**KMP 이동 거리**: 이동 거리 = 일치 접두부의 길이 − 최대 경계 너비 [U16-038] (B 10.4.2, p.475)

# 17 분할 정복

## 핵심 정의

**분할 정복**: **분할 정복 알고리즘**은 문제를 더 이상 나눌 수 없을 때까지 나누고, 나뉜 문제들을 각각 풀어 전체 문제의 답을 얻는 기법이다. [U17-001] (B 12.1.2, p.524)

**병합 정렬**: **병합 정렬**은 퀵 **정렬**과 같이 **분할 정복**에 기반한 **알고리즘**이다. [U17-009] (B 12.2, p.525)

## 원문 수행 시간 비교

| 항목 | 원문 설명 |
|---|---|
| **반복** 거듭 제곱 [U17-032] (B 12.3.1, p.533) | 처음 거듭 제곱 **알고리즘**은 Exponent 크기만큼 곱셈을 수행하여 Exponent에 대해 O(n)의 수행 시간이 소요된다. [U17-019] (B 12.3.1, p.533) |
| **분할 정복** 거듭 제곱 [U17-033] (B 12.3.1, p.534) | 지수를 반으로 나눠가는 거듭 제곱 **알고리즘**의 수행 시간은 $O(\log_2 n)$이다. [U17-022] (B 12.3.1, p.534) |

## 주요 수식·복잡도

**거듭 제곱의 분할**: $C^n = C^{n/2} \cdot C^{n/2}$ (n은 짝수)
$C^n = C^{(n-1)/2} \cdot C^{(n-1)/2} \cdot C$ (n은 홀수); 원문 두 경우의 **재귀 방정식**. [U17-029] (B 12.3.1, p.534)

# 18 동적 계획법

### 핵심 정의

**동적 계획법**: **동적 계획법**은 문제가 여러 단계의 **반복**되는 부분 문제로 이루어졌을 때 각 단계의 부분 문제 답을 기반으로 전체 문제의 답을 구하는 방법이다. [U18-001] (B 13.1.2, p.550)

**최적 부분 구조**: **최적 부분 구조**(Optimal Substructure)는 전체 문제의 최적해가 부분 문제의 최적해로부터 만들어지는 구조이다. [U18-006] (B 13.1.2, p.550)

**LCS**: **최장 공통 부분 수열**(LCS)은 여러 **공통 부분 수열** 중 가장 긴 것이다. [U18-021] (B 13.3, p.556)

### 원문 방향 비교

| 항목 | 원문 설명 |
| --- | --- |
| **분할 정복**의 방향 [U18-035] (B 13.1.2, p.550) | 위에서 아래로 문제를 쪼갠다(Top-Down). [U18-036] (B 13.1.2, p.550) |
| **동적 계획법**의 방향 [U18-037] (B 13.1.2, p.550) | 제일 작은 부분 문제부터 상위 문제로 풀어 올라간다(Bottom-Up). [U18-038] (B 13.1.2, p.550) |

### 주요 수식·복잡도

**동적 계획법 기반 피보나치**: O(n); 원문 직접 재귀 구현의 $O(2^n)$을 테이블 기반으로 재구성한 수행 시간. [U18-033] (B 13.2.1, p.552)

# 19 탐욕 알고리즘

## 핵심 정의

**탐욕 알고리즘**: **탐욕 알고리즘**은 각 단계의 부분 문제를 풀 때 근시안적으로 최적해를 구한다. [U19-001] (B 14.1, p.573)

**접두어 코드**: **접두어 코드**는 코드 집합의 어느 코드도 다른 코드의 접두어가 되지 않는 코드이다. [U19-025] (B 14.5.1, p.586)

**허프만 트리**: **허프만 트리**는 뿌리에서 잎에 이르는 경로로 기호의 **접두어 코드**를 표현하는 이진 트리이다. [U19-030] (B 14.5.2, p.588)

## 원문 탐욕 선택 비교

| 항목 | 원문 설명 |
| --- | --- |
| 거스름돈 [U19-039] (B 14.2, p.574) | **해 선택**에서 현재 고를 수 있는 가장 큰 단위의 동전 하나를 거스름돈에 추가한다. [U19-009] (B 14.2, p.574) |
| 크루스칼 [U19-040] (B 14.3, p.580) | 크루스칼의 **해 선택**은 가장 작은 가중치의 간선을 선택하는 일이다. [U19-017] (B 14.3, p.580) |
| **허프만 트리** [U19-042] (B 14.5.2, p.589) | **허프만 트리** 구성에서는 현재 빈도가 가장 작은 **노드** 2개를 선택한다. [U19-031] (B 14.5.2, p.589) |

# 20 백트래킹

## 핵심 정의

**백트래킹**: **백트래킹**은 여러 **후보해** 중 특정 조건을 충족시키는 모든 해를 찾는 **알고리즘**이다. [U20-003] (B 15.1.2, p.608)

**후보해**: **후보해**는 해가 될 가능성을 가진 부분해의 조합이다. [U20-004] (B 15.1.2, p.609)

## 원문 구현 비교

| 항목 | 원문 설명 |
| --- | --- |
| 미로 [U20-039] (B 15.2.2, p.616) | **이동**이 불가능하면 북→남→동→서 순으로 방향을 바꾸어 시도하고 어느 방향도 가능하지 않으면 이전 위치로 돌아간다. [U20-020] (B 15.2.2, p.616) |
| N개의 퀸의 저장 [U20-040] (B 15.3.2, p.629) | N개의 퀸 해를 담는 데 N개 요소의 **배열** 하나면 충분하며 인덱스는 행 번호, 값은 열 번호로 사용한다. [U20-002] (B 15.3.2, p.629) |
| 재귀 호출의 되돌아가기 [U20-041] (B 15.2.1, p.614) | 재귀 함수에서 **백트래킹** 조건을 만나 함수를 반환하면 상위 부분해의 함수로 제어가 돌아간다. [U20-015] (B 15.2.1, p.614) |

## 주요 수식·복잡도

**대각선 위협 조건**: |행1−행2| = |열1−열2|; 이 등식을 만족하면 두 퀸이 대각선으로 상호 위협한다. [U20-037] (B 15.3.2, p.632)